UNIVERSIDAD ICESI · DESARROLLO DE MICROSERVICIOS

Service Discovery y Circuit Breaker

Arquitectura resiliente para FutureX con Eureka y Resilience4j

26 de octubre de 2026

fx-discovery-server · fx-course-service · fx-catalog-service

Agenda

1. Arquitectura

Tres microservicios y Service Discovery con Eureka

2. El problema

Fallas en cascada cuando course-service cae

3. La solución

Cinco patrones de Resilience4j trabajando juntos

4. Demo en vivo

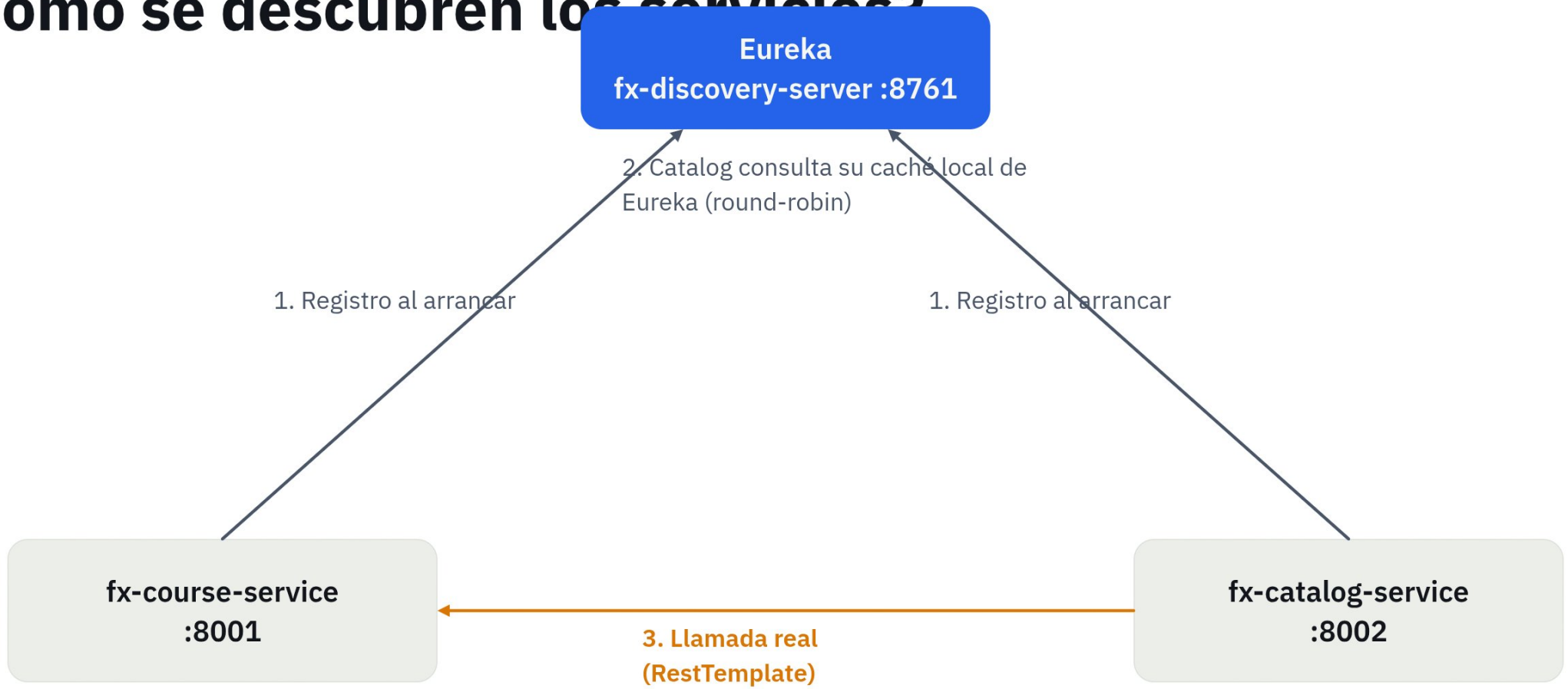
Evidencia medida y ejecución real con Docker y Postman

Tres microservicios Spring Boot

Servicio	Puerto	Rol
fx-discovery-server	8761	Servidor Eureka: el registro donde los servicios se anotan y se buscan
fx-course-service	8001	Proveedor: CRUD de cursos (JPA + H2 en memoria)
fx-catalog-service	8002	Consumidor: arma el catálogo pidiendo cursos al servicio anterior

Cada servicio corre en su propio contenedor Docker, orquestado con docker-compose.yaml

¿Cómo se descubren los servicios?



¿Por qué Eureka y no una URL fija?

⚠ URL fija

- Si course cambia de host o puerto, hay que recompilar el catálogo
- Solo funciona con una instancia
- No sabe si el destino sigue vivo

✓ Con Eureka

- El catálogo solo conoce el nombre lógico: “fx-course-service”
- Varias instancias con round-robin automático
- Eureka retira las instancias que dejan de mandar heartbeat

El problema: course-service se cae

fx-catalog-service responde HTTP 500

```
GET /catalog → 500
ResourceAccessException: Connection refused
```

Sin timeout, reintento ni fallback: cada request sigue intentando conectarse a un servicio muerto. En un sistema con muchos consumidores, eso agota hilos y conexiones — **falla en cascada**.

La solución: cadena de resiliencia

COURSE CAÍDO

Retry
3 intentos, 500ms

→

Circuit Breaker
corta si ≥50% falla

COURSE LENTO O SATURADO

RateLimiter
máx. 5 llamadas/s

→

TimeLimiter
corta a los 2s

→

Bulkhead
2 hilos + 1 en cola

Un circuito por dependencia: **courseService** protege los 3 endpoints del catálogo

Circuit Breaker: tres estados



Decisión de diseño: ¿dónde va el fallback?

⚠ Fallback en @CircuitBreaker

"Traga" la excepción antes de que Retry la vea → **Retry nunca reintenta**

✓ Fallback en @Retry (implementado)

Cada intento fallido alimenta al circuito; solo al agotar reintentos se degrada

Métrica (course caído)	Fallback en @CircuitBreaker	Fallback en @Retry
Tiempo de respuesta	0.002s	1.01s
Eventos de reintento	0	2 RETRY + 1 ERROR
Fallos registrados en el circuito	1	3

Resilience4j vs. Hystrix

Aspecto	Hystrix	Resilience4j
Estado	Mantenimiento desde 2018	Activo, soporte oficial Spring Boot 3
Diseño	Monolítico (todo vía HystrixCommand)	Modular: se usa solo lo necesario
Patrones	Circuit breaker, fallback, timeout, bulkhead	+ Retry, RateLimiter, TimeLimiter, Cache
Peso	Dependencias pesadas (Archaius, RxJava)	Ligero, sin dependencias pesadas

Hystrix no compila en Spring Boot 3 (Jakarta EE). Resilience4j es el reemplazo oficial recomendado por Netflix y Spring.

Evidencia real medida

1.03s

primera falla (3 intentos de Retry)

0.00s

circuito OPEN: fallback instantáneo

~20s

recuperación automática tras reiniciar course

```
11:37:09 NOT_PERMITTED
11:37:10 STATE_TRANSITION OPEN_TO_HALF_OPEN
11:37:10 SUCCESS · SUCCESS · SUCCESS
11:37:10 STATE_TRANSITION HALF_OPEN_TO_CLOSED
```

GET /actuator/circuitbreakerevents · ejecución real verificada en Docker Compose

Demo en vivo: qué vamos a mostrar

1. docker compose up --build -d → los 3 contenedores quedan healthy
2. Panel de Eureka (localhost:8761): los dos servicios en UP
3. Postman (entorno Docker Compose): catálogo responde normal, circuito CLOSED
4. docker compose stop course-service → Postman muestra el fallback y el circuito en OPEN
5. docker compose start course-service → recuperación automática (OPEN → HALF_OPEN → CLOSED)

Conclusiones

- Service Discovery: el catálogo nunca “supo” dónde vivía course; lo preguntó.
- Sin resiliencia, una caída se propaga en cascada.
- Con Resilience4j: se contiene, se reintenta, se corta y se limita la carga — sin esperar para siempre.
- Todo es observable en Actuator: no es una caja negra.

¿Preguntas?